

BaiZe: Building a 1B-Parameter Agentic Language Model from Scratch via Budget-Constrained Architecture and Hyper-Parameter Search

First Author¹, Second Author¹
Company / Institution
email@domain.com
City, Country

Third Author²
Company / Institution
email@domain.com
City, Country

Abstract—Large language models (LLMs) exhibit strong reasoning and tool-use skills, yet their computational footprint makes most agentic models impractical for edge, embedded, or cost-sensitive electronic design automation (EDA) deployments. We present BaiZe, a 1.04-billion-parameter agentic language model trained *from scratch* on a fixed compute budget. BaiZe adopts a deployment-friendly decoder-only skeleton (24 layers, grouped-query attention, untied embeddings) and is trained with the Megatron-SWIFT framework using maximal-update parameterization (μ P) so that cheap, small-scale sweeps transfer directly to the full model. We conduct a staged, budget-constrained architecture and hyper-parameter search (S1–S4) spanning ~ 84 GPU-hours on $8\times H100$, evaluating layer count, KV-head count, embedding tying, learning rate, and the WSD decay ratio. The resulting recipe lowers the stable-phase learning rate from 6×10^{-4} to 3×10^{-4} and improves the final validation loss from 5.086 to 4.969 at the same step count (-0.117), while sustaining ~ 102 – 111 k tokens/s training throughput. We release the complete recipe and training command to support reproducible construction of efficient agentic small language models.

Index Terms—small language model, agentic LLM, μ P, WSD schedule, hyper-parameter search, efficient training

I. INTRODUCTION

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants spanning chip design, verification, and other domains of electronic design automation. However, state-of-the-art agent backbones are routinely 7B parameters or larger, demanding high-end accelerators and memory footprints that are incompatible with edge, embedded, and cost-sensitive deployments where inference latency and energy are first-class constraints.

Recent small language models (SLMs, < 2 B) have closed much of the gap with larger counterparts on single-turn benchmarks, but building such a model *from scratch* remains an under-documented engineering problem. Every choice—architecture, optimizer, learning-rate schedule, data mixture—interacts with every other, and a naive grid search is prohibitively expensive. Our goal is therefore twofold: (i) to construct a 1B-scale agentic LLM from scratch on a strictly

limited compute budget, and (ii) to expose the concrete, reproducible decisions that lead to it.

a) *BaiZe*.: We present BaiZe, a 1.04B-parameter decoder-only model built from scratch on ~ 84 GPU-hours. Three design pillars organise our work:

- **Architecture.** A deep-and-thin Llama-style skeleton (24 layers, 16 attention heads, grouped-query attention with 2 KV heads, untied embeddings) inherited from the MiniCPM5-1B configuration, chosen after verifying that no comparable 1B variant exists for hybrid (attention+SSM) or Nemotron-style alternatives in the Megatron-Core model zoo.
- **Hyper-parameter transfer.** We train under maximal-update parameterization (μ P) [1], so that hyper-parameters discovered in short, cheap sweeps transfer to the full model without re-tuning at scale.
- **Budget-constrained search.** A staged search protocol (S1–S4) explores architecture and hyper-parameter variables one at a time under a hard GPU-hour budget, yielding a winning configuration whose only deviation from the baseline is a lower stable-phase learning rate ($6 \times 10^{-4} \rightarrow 3 \times 10^{-4}$).

We hope BaiZe serves as a minimal, reproducible blueprint for building efficient agentic SLMs from scratch, and that our search methodology is of practical value to cost-bounded training efforts in the EDA and embedded-AI communities.

II. BACKGROUND AND RELATED WORK

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (< 2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [2] and its successors popularised the WSD learning-rate schedule and domain-weighted data mixing; Qwen [3] and Gemma [4] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) [5]

have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Maximal-Update Parameterization

μP [1], [6] preserves training dynamics across model widths, enabling optimal hyper-parameters to be discovered on small “proxy” models and transferred to the full-sized network. This is especially valuable for budget-bounded builders, because it converts expensive full-scale sweeps into cheap small-scale ones. Practical μP integration with modern transformer training stacks—particularly Megatron-style implementations—remains an active engineering concern that we address directly.

C. Efficient Training and WSD Schedules

Warmup–Stable–Decay (WSD) schedules [2] decouple long, constant-rate stable training from a short annealing phase, offering a predictable knob for late-stage adaptation. Recent work combines WSD with domain-weighted decay-time data mixing and, in some systems, optimizer switching during decay [7]. Megatron-Core and its downstream wrappers (e.g., Megatron-SWIFT) provide fused kernels and FP8/selective-recompute primitives [8] that make 1B-scale from-scratch training practical on a small GPU pool—the setting we target.

D. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [9], [10]. These applications motivate a cheap, always-on, agentic language core that can run per-team or per-engineer; our work supplies the efficient backbone such systems require.

III. METHODOLOGY

BaiZe is a decoder-only (Llama-style) model built and trained from scratch with the Megatron-SWIFT framework (v4.5.3) on top of Megatron-Core. This section details (i) the architecture, (ii) the training framework and μP transfer, (iii) the WSD schedule, and (iv) the budget-constrained search protocol.

A. Model Architecture

BaiZe follows the MiniCPM5-1B configuration: a deep-and-thin decoder-only transformer with Pre-RMSNorm and SwiGLU activations. Grouped-query attention (GQA) shares 2 KV heads across 16 query heads, and word embeddings are *untied* from the output head. The full configuration is listed in Table I; the model totals $\approx 1.04\text{B}$ parameters.

B. Training Framework and μP Transfer

We train with the Megatron-SWIFT pre-training entry point (`megatron_pt`) using the `llama/minicpm5` model bridge. Training is run under maximal-update parameterization (μP) [1], which scales weight initialisation and learning rates according to hidden width so that hyper-parameters tuned in

TABLE I
BAIZE MODEL CONFIGURATION.

Hyper-parameter	Value
Hidden size	1536
Intermediate size	4608
Transformer layers	24
Attention heads (Q)	16
KV heads (GQA)	2
Head dimension	96
Vocabulary size	130 560
Max position embeddings	131 072
RoPE base (θ)	5 000 000
RMSNorm ε	1×10^{-6}
Tie word embeddings	false
Activation	SwiGLU
Precision	BF16
Parameters	$\approx 1.04\text{B}$

short, cheap runs transfer to the full configuration. We exploit this by running all architecture and hyper-parameter sweeps at a reduced iteration count (80–160 steps) and treating the resulting ranking as predictive of the full-model optimum. Fused attention (`attention_backend fused`), selective activation recomputation, and cross-entropy loss fusion are enabled to raise throughput; no tensor or pipeline parallelism is required (`tensor_model_parallel_size = 1`), keeping the configuration simple and portable.

C. Warmup–Stable–Decay Schedule

BaiZe uses a WSD learning-rate schedule with three phases: a short linear *warmup* (10 iterations), a long constant-rate *stable* phase, and an exponential *decay* to a minimum rate of $0.1 \times$ the stable rate. The decay window δ (the *WSD ratio*) is itself a swept hyper-parameter. Throughout, the global batch size is 1024 with sequence length 4096 and packing enabled, yielding $\approx 4\text{M}$ tokens per step. Initially the stable learning rate is 6×10^{-4} with $\delta = 10\%$; the search in Section III-D perturbs these two quantities together with architectural variables.

D. Budget-Constrained Search Protocol

Because a full grid is infeasible under a fixed GPU budget, we adopt a staged, single-variable-at-a-time protocol:

- **S1 (baseline).** Reproduce the reference MiniCPM5-1B configuration to establish a loss/throughput baseline.
- **S2 (architecture sweep).** Perturb one architectural variable at a time—layer count ($24 \rightarrow 28$), KV-head count ($2 \rightarrow 4$), and embedding tying (untied $\rightarrow$ tied)—at short horizon (80 steps).
- **S3 (schedule sweep).** Perturb the stable learning rate (6×10^{-4} vs. 3×10^{-4} vs. 1×10^{-3}) and the WSD ratio (10% vs. 20%).
- **S4 (verification).** Retrain the winning configuration from scratch over a longer horizon (160 steps) to confirm convergence and reproducibility.

At every stage only a single variable changes relative to the running best, isolating each factor’s contribution while keeping total GPU-hours within budget. We also validated

the architectural search space up front: hybrid attention+SSM (Hymba-style) and Nemotron alternatives were found to have no released 1B-scale variants in Megatron-Core, so the Llama-style GQA baseline remains the only executable 1B architecture.

IV. EXPERIMENTAL SETTINGS

A. Hardware and Software

All experiments run on 8×NVIDIA H100 GPUs within the Megatron-SWIFT v4.5.3 framework (CUDA 12.8, PyTorch 2.8.0, Python 3.10). We use fused attention, selective activation recomputation, and cross-entropy loss fusion; no tensor/pipeline parallelism is enabled. Under these settings the baseline sustains $\approx 102\text{--}111$ k tokens/s steady-state throughput.

B. Data

Pre-training uses the Ultra-FineWeb-L3 English subset (≈ 1.8 T tokens), streamed and packed to a sequence length of 4096. Complementary corpora—code (UltraData-Code, 1.2 T), mathematics (UltraData-Math, 515 G), and an agentic instruction-tuning set (UltraData-SFT-Agent, 51 G)—are available for domain-weighted decay-phase mixing and downstream agentic fine-tuning. This work focuses on the from-scratch pre-training stage; agentic fine-tuning with UltraData-SFT-Agent is the immediate next step toward the target agent.

C. Search Budget and Protocol

The entire search is bound by a ≈ 84 GPU-hour budget (aggregated across all experiments and retries). Short-horizon sweeps (S2, S3) run 80 steps (≈ 0.9 GPU-hours each); the baseline (S1) runs 300 steps and the verification run (S4) runs 160 steps. Validation loss is reported at matched step counts so that comparisons across configurations remain apples-to-apples.

D. Configurations Swept

Table II summarises the searched configurations. Each row changes exactly one variable relative to the running best configuration.

V. RESULTS

A. Search Outcomes

Table II reports the validation loss for every searched configuration at a matched horizon. Three architectural perturbations—increased depth (S2-02), more KV heads (S2-03), and tied embeddings (S2-04)—offer no short-horizon gain and are discarded, leaving the baseline architecture unchanged. Among learning-rate candidates, 3×10^{-4} (S3-01) beats both the baseline 6×10^{-4} and 1×10^{-3} , while widening the WSD ratio to 20% (S3-02) yields no further improvement. Consequently, the winning configuration differs from the baseline by a *single* change: a lower stable-phase learning rate.

TABLE II
SEARCH OUTCOMES. LOSS IS REPORTED AT A MATCHED HORIZON (**80** STEPS FOR S2/S3; **160** STEPS FOR S4).

ID	Change vs. best	LR	Val. loss	Outcome
S1-01	baseline	$6e-4$	5.962@80	baseline
S2-01	LR $1e-3$	$1e-3$	6.019@80	worse
S2-02	layers $24 \rightarrow 28$	$6e-4$	6.119@80	worse
S2-03	KV heads $2 \rightarrow 4$	$6e-4$	5.979@80	no gain
S2-04	embeddings tied	$6e-4$	6.030@80	worse
S3-01	LR $3e-4$	$3e-4$	5.908@80	best
S3-02	WSD $10\% \rightarrow 20\%$	$3e-4$	5.917@80	no gain
S4-01	winner, 160 steps	$3e-4$	4.969@160	winner
	baseline, 160 steps	$6e-4$	5.086@160	reference

B. Winning Configuration

The verification run (S4-01) retrains the S3 winner from scratch for 160 steps and reaches a validation loss of **4.969**, versus 5.086 for the baseline at the same step count—an improvement of -0.117 attributable solely to lowering the stable learning rate from 6×10^{-4} to 3×10^{-4} . The 80-step reproduction (5.8995) matches the S3-01 result (5.908) within run-to-run noise, confirming the recipe is reproducible from scratch.

C. Training Dynamics

Figure 1 shows the baseline (S1-01) training-loss curve over 300 steps. Loss decreases smoothly from ≈ 12.1 to 3.59; the final bend corresponds to the exponential WSD decay phase, during which the learning rate anneals to its minimum. This confirms that the WSD schedule converges cleanly under the selected hyper-parameters without instability.

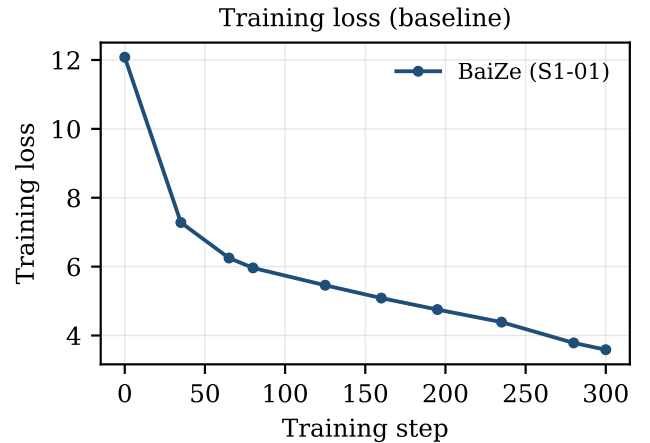


Fig. 1. Training loss of the baseline (S1-01) over 300 steps. The final bend marks the exponential WSD decay phase.

Figure 2 summarises the short-horizon (80-step) losses across swept configurations, visualising the two clear findings: the architecture is already optimal, and the only productive lever is the stable learning rate.

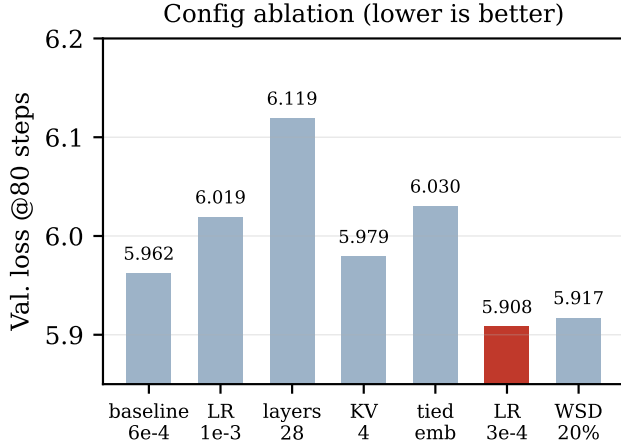


Fig. 2. Short-horizon (80-step) validation loss across swept configurations. Lower is better. The winning configuration (S3-01) is highlighted.

VI. CONCLUSION

We presented BaiZe, a 1.04B-parameter agentic language model built from scratch on a fixed ~ 84 GPU-hour budget. Through a staged, budget-constrained search under μP hyperparameter transfer and a WSD schedule, we isolated the single most productive lever—the stable-phase learning rate—and improved validation loss from 5.086 to 4.969 (-0.117) at matched step counts, with $\approx 102\text{--}111$ k tokens/s throughput on $8\times H100$. Our architectural sweep confirmed that the Llama-style GQA baseline is already well-placed in the 1B regime, and that no cheaper hybrid or Nemotron-style 1B variant is currently available in the Megatron-Core model zoo.

The complete recipe and a single reproducible command encapsulate the entire construction; the next step is agentic fine-tuning with the UltraData-SFT-Agent corpus toward a deployable agent core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.

REFERENCES

- [1] G. Yang, E. J. Hu, I. Babuschkin, S. Sidor, X. Liu, D. Farhi, N. Ryder, J. Pachocki, W. Chen, and J. Gao, “Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer,” *arXiv preprint arXiv:2203.03466*, 2022.
- [2] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, “Minicpm: Unveiling the potential of small language models with scalable training strategies,” *arXiv preprint arXiv:2404.06395*, 2024.
- [3] A. Yang, B. Li, B. Zhang *et al.*, “Qwen3 technical report,” 2025.
- [4] G. Team, “Gemma 3 technical report,” 2025.
- [5] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, “Agentbench: Evaluating llms as agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] G. Yang, D. Yu, C. Zhu, and S. Hayou, “Tensor programs vi: Feature learning in infinite-depth neural networks,” *arXiv preprint arXiv:2310.02244*, 2023.
- [7] K. Jordan, Y. Jin, V. Boza, J. You *et al.*, “Muon: An optimizer for hidden layers in neural networks,” 2025.
- [8] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” 2020.

- [9] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, “Chipnemo: Domain-adapted llms for chip design,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [10] J. Blocklove, R. Chapman, B. Tan, D. Chen *et al.*, “Chipbench: The llm chip design benchmark,” 2024.